UNIVERSIDAD COMPLUTENSE MADRID | nticmaster

Programación avanzada en Python

Guía de estudio

Estructura del Curso y Materiales

En el Campus Virtual encontrarás un archivo comprimido que contiene los materiales del curso organizados en carpetas del **tema 1 al 8**. Dentro de cada tema se incluyen varios documentos de tipo **Notebook (.ipynb)**, secuenciados numéricamente para indicarte el orden recomendado de lectura y trabajo. Estos notebooks deben ser abiertos y ejecutados mediante **Jupyter Notebook**.

Tema 1. Configuración del Entorno de Trabajo

Antes de comenzar, es fundamental preparar correctamente tu entorno de desarrollo local. Para ello, en el **Tema 1** dispones de dos recursos clave:

1. **Guía de instalación de PyCharm:** Encontrarás un documento en formato **HTML** que contiene las instrucciones paso a paso para instalar y configurar el IDE PyCharm. Por supuesto puedes usar el IDE que más te guste.
2. **Guía de Entornos Virtuales:** Un documento explicativo sobre la importancia y el funcionamiento de los entornos virtuales en Python.

Importante: Debéis prestar **especial atención a la sección 4** de este documento, donde se explica cómo crear y gestionar un proyecto utilizando el gestor de paquetes y entornos **uv**. Os proporcionamos también **un video que muestra los pasos a seguir para crear un proyecto** usando uv, crear el entorno virtual asociado al proyecto, instalar paquetes, ejecutar scripts, lanzar jupyter lab, etc. Es recomendable que visualices el video antes de continuar.

Antes de pasar al siguiente tema, copia todos los temas proporcionados en el campus virtual en la carpeta src de tu proyecto. Quedará algo así:

```
src/
```

```
├── t1_ide/
└── t2_basico/
└── …

└── t8_tests_paquetes/
```

Tema 2. Elementos Básicos de Python

En este tema se recogen los conceptos fundamentales del lenguaje (tipos de datos primitivos, listas, tuplas, diccionarios, instrucciones de control de flujo, etc.). Este contenido se supone ya conocido por vosotros, pero lo incluimos en el curso a modo de recordatorio para asentar las bases antes de avanzar a contenidos más complejos.

Cómo trabajar con los Notebooks

Para ejecutar y modificar los notebooks de este y del resto de los temas del curso, tendrás que lanzar **Jupyter Lab**.

Puedes lanzarlo directamente desde tu proyecto en PyCharm siguiendo estos pasos:

1. Abre una **terminal** dentro de PyCharm.
2. Comprueba que tu **entorno virtual está activo**.
3. Ejecuta el siguiente comando:

```
uv run jupyter lab
```

Una vez ejecutado, se abrirá la interfaz de Jupyter Lab en tu navegador, donde podrás acceder a la carpeta `src/t2_basico/` y trabajar de forma interactiva con los archivos .ipynb.

Puede que el documento más importante sea: **s2_funciones.ipynb.**

Tema 3. Funciones Avanzadas

Aquí se comienza con la **documentación del código mediante Docstrings.** Pretendemos convencerte de la importancia de documentar de manera estándar el código que escribas. Utilizaremos *docstrings* para redactar descripciones claras de lo que hace cada función, especificando el propósito de sus parámetros y el valor de retorno esperado, lo que resulta fundamental para facilitar el mantenimiento en proyectos de desarrollo.

En segundo lugar estudiaremos el uso de anotaciones de tipo (*type hints*) para indicar explícitamente los tipos de datos de entrada y salida en las funciones. Analizaremos cómo esta práctica ayuda a prevenir errores en tiempo de desarrollo, mejora el soporte

y autocompletado en los IDEs y hace que el código sea mucho más legible y autocontenido. Aquí también vamos a tratar de convencerte de que tipar no es una pérdida de tiempo.

Por último, introducimos las funciones de orden superior, un pilar del paradigma funcional y que no se suelen estudiar en cursos de Python (te van a encantar). Veremos cómo se pueden usar funciones pasándolas como argumentos a otras funciones, retornándolas como resultados o aplicando herramientas clásicas del lenguaje para escribir un código más flexible y reutilizable.

Tema 4. Procesamiento de Archivos y Expresiones Regulares

En este tema presentamos cómo gestionar el ciclo de vida de archivos en disco (.txt, .csv, .json) mediante el uso seguro de bloques with y funciones de lectura/escritura (open, read, write), garantizando una correcta liberación de recursos. Esto permitirá evitar pérdidas de información o archivos corruptos durante la ejecución de tus programas.

Por otra parte, haremos uso de expresiones regulares mediante el módulo re para el procesamiento de datos no estructurados. Se utilizarán herramientas como search o match para la validación de patrones, extracción y sustitución de texto.

Tema 5. Programación Orientada a Objetos

En este tema introducimos la **Programación Orientada a Objetos (POO)** para que aprendas a estructurar tus programas de una forma mucho más organizada y reutilizable. Si ya tienes nociones de POO, verás que aquí presentamos los mismos conceptos, y cómo se expresan en Python. Empezamos viendo cómo definir nuevas clases y crear objetos a partir de ellas, usando el método constructor `__init__` y el famoso parámetro `self` para manejar los datos propios de cada instancia. A lo largo de los notebooks, recorreremos los pilares clave de la POO —como la herencia, el encapsulamiento y el polimorfismo—, lo que te permitirá modelar problemas reales mediante código limpio, ordenado y fácil de mantener.

Tema 6. Control de Errores y Excepciones

En este tema veremos cómo hacer que los programas no se "rompan" cuando algo sale mal. Veremos cómo capturar los errores típicos de ejecución (como un `ValueError`,

`TypeError` o `KeyError`) usando los bloques `try`, `except`, `else` y `finally`. Además, veremos cómo lanzar errores cuando lo necesites usando `raise` y hasta cómo crear excepciones personalizadas para tus proyectos.

Tema 7. Análisis de Datos con Pandas y Polars

En este tema estudiaremos la librería **Pandas**, el estándar para la manipulación de datos en Python, e introduciremos **Polars** como una novedad. Mientras que Pandas es ideal para análisis exploratorios y es una herramienta ya consolidada en el ecosistema, Polars destaca por su rendimiento: procesa los datos en paralelo y consume mucha menos memoria. Veremos un poquito de ambas herramientas para que aprendas cuándo conviene usar la versatilidad de Pandas y cuándo la velocidad y eficiencia de Polars.

Tema 8. Pruebas Unitarias y Organización de Paquetes

En este tema nos centramos en garantizar la calidad y la estructura de los proyectos de Python. En la primera parte, veremos cómo comprobar que el código funciona correctamente escribiendo pruebas unitarias con `pytest` y `doctest`, analizando las aserciones y midiendo la cobertura de los tests. A continuación, veremos cómo organizar el código en módulos y paquetes mediante archivos `__init__.py`, cómo gestionar las dependencias del proyecto usando el estándar moderno `pyproject.toml` y cómo empaquetar y construir tu proyecto de forma sencilla utilizando `uv build`.

Y para terminar…

Tema 9. Extracción de Datos en Web / Scraping

En este último tema mostramos cómo extraer información de forma automatizada desde páginas web (mostramos un ejemplo bastante fácil). Veremos cómo realizar peticiones HTTP con requests, analizar la estructura HTML e inspeccionar elementos para extraer datos utilizando librerías como BeautifulSoup. Creo que estos ejemplos te resultarán de gran ayuda para implementar la práctica del curso.